Захист лабораторної роботи №1 (Варіант 14)

Студент: Мороз Максим Тарасович | **Група:** KI-404 | **Варіант:** 14
Стек варіанта 14: Python 3.11+, віртуальне оточення `venv`, фреймворк `pytest`, скрипти запуску `ci.bat` / `ci.sh`, CI/CD GitHub Actions (Windows, Linux, macOS).
Репозиторій: csad-lab1 | **PR:** #1 lab1/finish

---

1. Суть завдання варіанта 14

Що вимагалося зробити: Створити спеціалізованого AI-агента для ролі **Build Engineer**, який автоматично генерує та налаштовує: 1. Python-проєкт (`src/app.py` з функціями привітання та додавання). 2. Ізольоване середовище `venv` та залежності `requirements.txt` (`pytest ⩾ 8.0.0`). 3. Модульні тести `tests/test_app.py`. 4. Кросплатформні скрипти складання: `scripts/ci.bat` (Windows) та `scripts/ci.sh` (Linux/macOS). 5. CI-пайплайн GitHub Actions (`.github/workflows/ci.yml`) із матрицею з 3 ОС, який запускає **виключно** ці скрипти.

---

2. Структура AI-агента (Архітектура)

Які файли реалізують агента?

- **Маніфест (`.github/agents/build-engineer.agent.md`):** Описує роль Build Engineer, правила безпеки (заборонено видаляти файли, заборонено `force push`, заборонено секрети в Git) та критерій готовності (всі тести зелені на 3 ОС).
- **3 Навички (`.github/skills/`):**
- `project-scaffold`: генерація `src/app.py` та `.gitignore`.
- `build-and-test`: налаштування `venv`, `requirements.txt`, тестів `test_app.py` та скриптів `ci.bat` / `ci.sh`.
- `github-actions`: створення workflow `.github/workflows/ci.yml`.
- **Команди (`.github/prompts/`):**
- `git-init` — ініціалізація Git та `.gitignore`.
- `create-project` — виклик скіла `project-scaffold`.
- `create-build` — виклик скіла `build-and-test`.
- `create-actions` — виклик скіла `github-actions`.
- `check` — **неруйнівна** перевірка структури, YAML і запуск тестів (нічого не змінює).
- `init` — **оркестратор**, який запускає команди по черзі за принципом **Fail-Fast**.

Що таке Fail-Fast в оркестраторі?

Якщо будь-який крок завершується з помилкою (код $\ne 0$), оркестратор **негайно перериває процес** і не виконує наступні дії. Помилка не приховується.

---

3. Скрипти збірки: `ci.bat` та `ci.sh`

Чому в `ci.yml` викликаються тільки скрипти, а не команди напряму?

Вимога методички: **локальна відтворюваність** та **єдине джерело істини** (Single Source of Truth). На комп'ютері розробника і на сервері GitHub Actions запускається один і той самий файл. Зміна логіки збірки робиться в одному місці — у скрипті.

Як працює `scripts/ci.bat` (Windows)?

```
@echo off
setlocal
set "ROOT_DIR=%~dp0.."
cd /d "%ROOT_DIR%"
if not exist .venv ( python -m venv .venv )
call .venv\Scripts\activate.bat
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pytest tests/test_app.py
if errorlevel 1 exit /b %errorlevel%
python src/app.py
exit /b %errorlevel%
```

- `%~dp0..` — визначає кореневу папку проєкту відносно розташування скрипта.
- `cd /d` — перехід у корінь проєкту (прапорець `/d` дозволяє змінювати диск).
- `call .venv\Scripts\activate.bat` — **обов'язково з** `call`, інакше CMD завершить роботу після активації venv і не виконає запуск тестів.
- `if errorlevel 1 exit /b %errorlevel%` — перевірка коду повернення pytest. Якщо тест впав — аварійний вихід, програма `src/app.py` не запускається (Fail-Fast).

Як працює `scripts/ci.sh` (Linux/macOS)?

```
#!/usr/bin/env bash
set -euo pipefail
ROOT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$ROOT_DIR"
if [ ! -d .venv ]; then python3 -m venv .venv; fi
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pytest tests/test_app.py
python src/app.py
```

- `set -euo pipefail` — режим суворої безпеки:
- `-e` — вихід при будь-якій помилці (Fail-Fast).
- `-u` — вихід при зверненні до неоголошеної змінної.
- `-o pipefail` — фіксація помилки всередині будь-якої ланки пайплайну.

4. Дві реальні проблеми та їх фікси (Troubleshooting)

Проблема на GitHub Actions	Причина	Як виправлено
1. **`Permission denied`** **(exit code 126)** на Linux/macOS при виклику `ci.sh`	Файл створювався у Windows (NTFS), тому Git зберіг його з правами `100644` (не виконуваний).	Встановлено біт виконання в індексі Git: `git update-index --chmod=+x scripts/ci.sh` Права стали `100755`.
2. **`ModuleNotFoundError: No module named 'src'`** при виконанні тестів	Команда `pytest tests/test_app.py` не додавала корінь проєкту в `sys.path`.	Запуск змінено на виконання модуля Python: `python -m pytest tests/test_app.py` (Python автоматично додає робочу папку в `sys.path`).

5. GitHub Actions Workflow (`ci.yml`)

- **Матриця з 3 ОС:** `yaml strategy: fail-fast: false matrix: os: [ubuntu-latest, windows-latest, macos-latest] runs-on: ${{ matrix.os }}`
- **Чому `fail-fast: false` в CI:** Якщо на Windows тест впаде, раннери Ubuntu та macOS продовжать роботу. Це дозволяє одразу бачити: помилка кросплатформна чи специфічна для конкретної ОС.
- **Розгалуження запуску:**
- `if: runner.os ≠ 'Windows'` $\rightarrow$ виконує `bash ./scripts/ci.sh`
- `if: runner.os = 'Windows'` $\rightarrow$ виконує `cmd .\scripts\ci.bat`
- **Тригери:** `push` у `develop` та `feature/*`, `pull_request` у `develop`.

6. Що знаходиться в `.gitignore` і чому?

- `.venv/` — локальне віртуальне середовище (займає багато місця, прив'язане до конкретної ОС та абсолютних шляхів).
- `__pycache__/` та `.pytest_cache/` — скомпільований байткод Python та кеш тестів.
- Тимчасові файли та логи. Секрети в репозиторій потрапляти не повинні.

7. Демонстрація на захисті (Покроково)

Крок 1. Запуск робочого проєкту

У терміналі виконати:

```
.\scripts\ci.bat
```

Результат: Активується venv, оновлюється pip, проходять 3 тести (`3 passed in 0.01s`), друкується привітання та сума 5.

Крок 2. Перевірка цілісності (`check`)

```
python -c "import yaml; yaml.safe_load(open('.github/workflows/ci.yml'))"
python -m pytest tests/test_app.py
```

Результат: Синтаксис YAML валідний, тести пройдені, жоден файл не змінено.

Крок 3. Реакція на НАВМИСНО ЗЛАМАНУ ПЕРЕВІРКУ *(Вимагає викладач!)*

1. У файлі `tests/test_app.py` змінити: `python assert add(2, 3) == 999 # замість 5`
2. Запустити в терміналі: `cmd .\scripts\ci.bat`
3. **Що сказати викладачу:**

   > *«Тест `test_add` провалився. Скрипт повернув код 1 і спрацював принцип Fail-Fast: рядок `python src/app.py` взагалі не викликався. Збірка аварійно зупинилася.»*

4. Повернути `assert add(2, 3) == 5`.